TrustTrail: Accountable, Purpose-Bound Attribute Release for Privacy-Aware Federated Identity Management

Vijender Yadav (23BCS132), Ansh Hardaha (23BCS016), and Sourabh (23BEC055)

Abstract

Single sign-on lets a student reach the library, the learning system and the examination portal with one login, but a typical deployment gives every service the same identifier and a fixed set of personal claims. Services can link a student across systems, nothing ties a release to a stated purpose or deletion date, and the student has no record of who received what. Werner, Westphall and Westphall surveyed privacy strategies for cloud identity management in 2017 and organised them into nine characteristics. Their own tables show that accountability and use limitation are the least covered: 2 and 5 of the 23 surveyed proposals address them, and none of the authors' four follow-up works do. This paper presents TrustTrail, a disclosure layer that sits beside an unmodified OpenID Connect provider (Keycloak 26). Each service receives a different pairwise pseudonym and, through a custom protocol mapper, no shared session identifier. Keycloak caps which of four disclosure profiles a service may obtain, and TrustTrail alone decides which attribute values that profile contains. Before receiving data, a service signs a purpose declaration that names the attributes, the purpose and a deletion date; TrustTrail refuses any release the declaration does not cover and returns a signed receipt. Every release and every refused rule break is written to a SHA-256 hash-chained ledger, whose batches are summarised as Merkle roots and anchored in an Ethereum smart contract, so that edits, deletions and truncation by the operator can be detected without placing personal data on chain. Three violation rules lower a service's trust score, and each step down automatically removes a disclosure profile from its Keycloak client. In our evaluation on a local deployment, the privacy layer added a median of 3.6 ms (22%) to a login. All six tamper attacks we tried were detected by at least one of the three checks, all 45 scripted violations were detected with the correct rule and no false positives over 90 honest releases, and anchoring a batch cost about 77,000 gas regardless of its size. We discuss what TrustTrail cannot guarantee, most importantly that a service deletes data after it has received it.

Index Terms

Federated identity management, privacy, OpenID Connect, pairwise pseudonyms, purpose limitation, accountability, tamper-evident logging, Merkle tree, blockchain anchoring, trust management.

I. Introduction

A UNIVERSITY runs many online services for its students: a digital library, a learning management system (LMS), an examination portal, and often third-party applications as well. Identity federation and single sign-on (SSO) made this manageable: the student logs in once at an identity provider (IdP), and each service, called a relying party, accepts the IdP's assertion instead of keeping its own password database [1], [2]. Protocols such as SAML 2.0 [3] and OpenID Connect (OIDC) [4] on top of OAuth 2.0 [5] are mature, and open-source IdPs such as Keycloak [6] make SSO cheap to deploy.

SSO solves the login problem. It does little for privacy, and in some ways it makes privacy worse. The project started from a simple question: what does a typical campus SSO configuration actually send to each service? The answer shows three problems.

Over-release. The easiest configuration gives every client the same set of claims: name, e-mail address, roll number, department and year. The library needs to know that the borrower is a current student. It does not need the student's name, and it certainly does not need the roll number. Studies of permission systems show that applications routinely ask for more than they need when nothing stops them [7], [8].

The authors are B.Tech. students (7th semester) at the Indian Institute of Information Technology Dharwad, Karnataka, India.

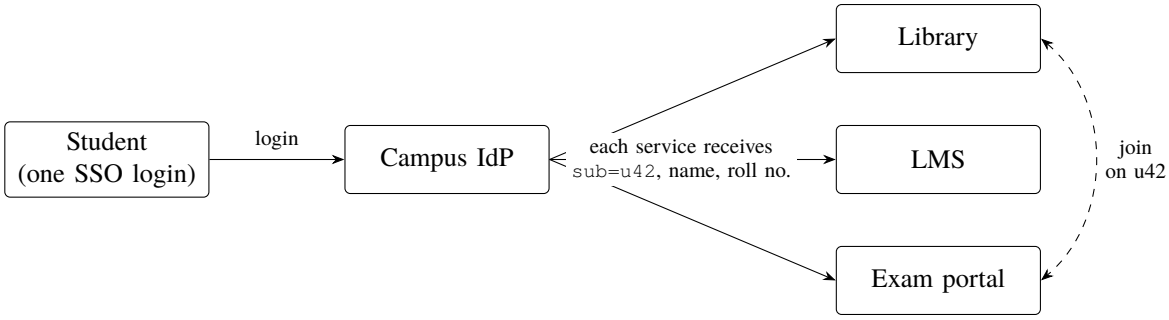


Fig. 1. The problem in a default SSO deployment. Every service receives the same identifier and the same personal claims, whatever it needs, so services can join their records; the student has no record of the releases.

Linkability. If the IdP gives every service the same subject identifier, or the same e-mail address, any two services can join their records. Even when identifiers differ, a session identifier shared across clients in one SSO session can link them, as we show in Section **??**. Unlinkability is one of the three privacy properties that Birrell and Schneider identify for federated identity [1], and it is the one that a default deployment gives up first.

No record and no recourse. After a release, the student cannot find out which service received which attribute, for what purpose, or until when. If a service later asks for something it never said it needed, nobody notices, and the service keeps the same access as before. Data protection law expects the opposite. The EU General Data Protection Regulation requires that personal data be collected for specified purposes, limited to what is necessary, kept no longer than necessary, and that the controller be able to demonstrate compliance [9]. India's Digital Personal Data Protection Act, 2023 likewise ties processing to a specified purpose and requires erasure once that purpose is no longer served [10]. These are statements about what should happen. A campus IdP gives no evidence of whether it did.

Our starting point was the survey by Werner, Westphall and Westphall, "Cloud identity management: A survey on privacy strategies" [11]. The survey proposes a taxonomy of nine privacy characteristics for identity management (transparency, controllability, minimization, accountability, data quality, use limitation, user-friendliness, trust and obfuscation), maps 23 published proposals and six European research projects against it, and lists open challenges. Reading its tables closely, we found that the two characteristics that matter most for the third problem are also the least covered. Only 2 of the 23 proposals address accountability and only 5 address use limitation; the four works the authors themselves present as solutions in progress address neither [11, Tables 2–3]. The survey also names "ensure the implementation of policies" as an open challenge and observes that several surveyed proposals were never implemented or validated. Section III analyses the survey in detail.

This paper presents *TrustTrail*, a system that addresses these gaps for an OIDC federation and that we built and evaluated end to end. TrustTrail keeps the IdP unmodified in its protocol behaviour and adds a disclosure layer beside it. Its design follows one loop:

$$\textit{signed promise} \rightarrow \textit{ledger record} \rightarrow \textit{violation detected} \rightarrow \textit{trust lowered} \rightarrow \textit{less data released.}$$

A service first signs a purpose declaration that names the attributes it wants, why it wants them and when it will delete them. TrustTrail releases attribute values only under such a declaration and only up to the disclosure profile that the service's current trust tier allows. Every release, and every refused request that breaks a rule, becomes an entry in a hash-chained ledger. Batches of ledger entries are summarised as Merkle roots and anchored in an Ethereum smart contract, so the history cannot be quietly rewritten even by the operator. A service that breaks a rule loses trust, and the IdP stops granting it the more revealing profiles.

A. Research questions

We framed the work around five questions.

RQ1 Can a federation give each service only the attributes of an agreed disclosure profile, and stop services from linking a student across services, while the IdP keeps speaking standard OIDC?

RQ2 Can every release be bound to a purpose and a deletion date that the service has signed, so that a service cannot later deny what it promised?

RQ3 Can the record of releases be made tamper-evident against an insider who controls the database, including the operator, without putting personal data on a public ledger?

RQ4 Can rule-breaking that the disclosure layer can observe automatically reduce what the offending service receives in future?

RQ5 What do these mechanisms cost in login latency and in blockchain fees?

B. Contributions

1) A gap analysis of Werner et al. [11] that uses the survey's own tables to show where its taxonomy is least covered, and that maps each of its five open challenges to a concrete design requirement (Section III).
2) A design that splits disclosure between the IdP and a separate release service: Keycloak decides *which* profile a service may obtain, using per-service scope assignment as a tier cap, pairwise subject identifiers and a custom protocol mapper that removes the shared session identifier; TrustTrail decides *what* the profile contains and is the only path for attribute values (Sections VI and **??**).
3) Signed purpose declarations and release receipts based on JSON Web Signatures [12], which bind every release to the attributes, purpose and deletion date that the service promised, and give the student portable, verifiable evidence of each release.
4) An accountability chain made of a hash-chained ledger, per-release receipts and Merkle-root anchoring in an Ethereum contract that only accepts contiguous batches, with a verification procedure that a student can run using only a downloaded receipt and the chain.
5) A dynamic trust model in which three observable violation rules lower a service's score and its tier, and the tier is enforced both at the IdP (login is refused) and at the release service (already issued tokens get the lower cap).
6) An open implementation (Keycloak 26.8.1, Python 3.12/FastAPI, Solidity 0.8.28, Hardhat 3) with 149 Python tests and 6 contract tests, and an evaluation of latency, tamper detection, violation detection and anchoring cost (Sections **??** and **??**).

We are careful about what TrustTrail does not do. It records what a service promised and what it received, and it refuses releases that break the promise. It cannot see what a service does with data afterwards, so it cannot prove deletion. The IdP still sees every login. Section **??** discusses these and other limits.

C. Organisation

Section II introduces the protocols and primitives that TrustTrail builds on. Section III analyses the base survey and derives the gaps we address, and Section IV reviews related work. Section V states the threat model and design goals. Sections VI and **??** present the architecture and the detailed design, and Section **??** the implementation. Section **??** reports the evaluation. Sections **??**, **??** and **??** discuss limitations, future work and conclusions.

II. Background

This section introduces the protocols, cryptographic building blocks and privacy principles that the rest of the paper relies on. Readers familiar with OIDC and Merkle trees can skip to Section III.

A. Identity management, federation and single sign-on

Identity management (IdM) covers how identities are created, authenticated, described by attributes and eventually removed. In a federation, several organisations agree to trust each other's identity assertions. One party, the identity provider (IdP), authenticates the user and issues an assertion; the service provider (SP), or relying party (RP) in OIDC terms, accepts the assertion and makes its own authorisation decision [1], [11]. Single sign-on (SSO) follows: once the user has a session at the IdP, further services can be reached without logging in again.

Federations differ in how much the members trust each other. Ferdous and Poet distinguish *fully trusted*, *semi-trusted* and *untrusted* entities in dynamic SAML federations, depending on whether a legal agreement backs the relationship [2]. TrustTrail reuses these three names for its service tiers and adds a fourth, *blocked*, for services that have lost trust through violations.

From a privacy point of view, the IdP is in a powerful position. It knows every user's attributes and sees every login to every service. Birrell and Schneider characterise federated IdM systems by three properties that users may want [1]: *undetectability* (others cannot tell that a transaction happened), *unlinkability* (transactions cannot be connected to each other or to the user) and *confidentiality* (the user controls who learns their attributes). Pfitzmann and Hansen give the standard terminology for anonymity, unlinkability and pseudonymity [13]. A *pseudonym* is an identifier other than the user's real name; it is *relationship-specific* when the user has a different pseudonym for each communication partner, which prevents partners from linking the user by identifier.

B. OAuth 2.0 and OpenID Connect

OAuth 2.0 [5] lets a client obtain an *access token* from an authorisation server in order to call a protected API. The client asks for *scopes*, which name the permissions it wants, and the server decides which scopes to grant. Two OAuth flows matter here. In the *authorization code* flow, the user's browser is redirected to the server, the user authenticates, and the client exchanges a one-time code for tokens over a back channel. In the *client credentials* flow, a client authenticates as itself, with no user involved; TrustTrail uses it to authenticate services that register their keys.

OpenID Connect [4] adds authentication on top of OAuth. The server, now an OpenID Provider, issues an *ID token*: a signed JSON Web Token (JWT) [14] whose claims state who the user is (`sub`), who issued the token (`iss`), for which client it is meant (`aud`) and when it was issued and expires. Additional user attributes, such as name or e-mail address, can be placed in the ID token, the access token, or returned from a userinfo endpoint, depending on the requested scopes and the provider's configuration. Keycloak [6] controls this per client through *client scopes* and *protocol mappers*. A client scope bundles claims; it can be assigned to a client as a *default* scope (always granted) or an *optional* scope (granted only when requested). A protocol mapper is a plug-in that adds, changes or removes claims in the tokens that a client receives.

1) Pairwise subject identifiers: OIDC defines two subject identifier types [4, Sec. 8]. With the *public* type, every client receives the same `sub` for a user. With the *pairwise* type, each client, or more exactly each *sector* of clients, receives a different `sub`, so clients cannot correlate the user by identifier. The specification suggests computing the pairwise value as a hash of the sector identifier, the local account identifier and a secret salt. Keycloak's SHA-256 pairwise mapper does this and then formats the 32-byte digest as a name-based (version 3) UUID; the sector is the host name of the client's redirect URIs unless a sector identifier URI is configured. Pairwise identifiers are also recommended by NIST's federation guidelines [15].

2) Session identifiers: Keycloak adds a session identifier claim, `sid`, to the tokens it issues. All clients that a user reaches within one SSO session receive the same `sid` value. The authorisation response can also carry a `session_state` parameter derived from the session. As Section **??** shows, these values defeat pairwise identifiers: two services that compare `sid` values can link the two pseudonyms of the same student.

C. JSON Web Signatures and keys

A JSON Web Signature (JWS) [12] protects a payload with a digital signature or MAC. Its *compact serialisation* is three base64url strings separated by dots: a protected header, the payload and the signature. The header names the algorithm (`alg`), may name the signing key (`kid`) and may state a media type (`typ`) so that a JWS meant for one purpose is not accepted for another. TrustTrail uses two algorithms from the JSON Web Algorithms specification [16]: ES256 (ECDSA on curve P-256 with SHA-256) and RS256 (RSASSA-PKCS1-v1_5 with SHA-256). Public keys are exchanged as JSON Web Keys (JWK) [17]. The JWK thumbprint [18] is the SHA-256 hash of a canonical JSON object made of the key's required members in lexicographic order; it gives a key a stable identifier that is derived from the key itself, which TrustTrail uses as the `kid`.

A signed JWS gives *non-repudiation* in a practical sense: if the verifier keeps the JWS, it can later show a third party that the holder of the private key signed exactly this payload. TrustTrail relies on this property for purpose declarations, which services sign, and for release receipts, which TrustTrail signs.

D. Hash chains and tamper-evident logs

A cryptographic hash function such as SHA-256 maps any input to a fixed-size digest; it is infeasible to find two inputs with the same digest or to change an input without changing its digest. A *hash chain* links records by

including the digest of the previous record in each new record. Haber and Stornetta used this idea to timestamp digital documents [19], and Schneier and Kelsey applied it to audit logs, so that an attacker who takes over a logging machine cannot undetectably change entries written before the takeover [20]. A chain alone has a known weakness: whoever controls the whole log can rewrite an entry and recompute every later hash, or cut off the newest entries. Detecting that requires some value that the attacker cannot change, such as a hash that has been published elsewhere or handed to a third party [21], [22].

E. Merkle trees

A Merkle tree [23] hashes a list of items in pairs, then hashes the results in pairs, until one *root* hash remains. The root commits to every item and its position. To prove that an item is in the tree, it is enough to give the hashes of its siblings on the path to the root, which takes $\lceil \log_2 n \rceil$ hashes for n items; the verifier recomputes the root and compares. Certificate Transparency (CT) uses Merkle trees for public, append-only logs of TLS certificates [24]. Its specification, RFC 6962 [25] (updated by RFC 9162 [26]), prefixes leaf inputs with the byte `0x00` and interior-node inputs with `0x01`. This domain separation stops an attacker from presenting an interior node as a leaf. TrustTrail follows these conventions.

F. Blockchains, smart contracts and anchoring

A blockchain is a replicated ledger in which blocks of transactions are linked by hashes and agreed on by many independent nodes, so that a confirmed transaction is very hard to remove or change [27]. Ethereum [28] adds *smart contracts*: programs stored on the chain whose functions are called by transactions. A contract can keep state, emit *events* that clients can search, and reject calls that break its rules. Every operation costs *gas*, paid for in the chain's currency; writing to contract storage is among the most expensive operations. Contracts are usually written in Solidity [29] and tested on a local development chain such as Hardhat [30] before deployment to a public test network such as Sepolia or to the main network.

Anchoring uses a blockchain as the external, hard-to-change place where a log's hash is published. Instead of writing each record, a system periodically writes one Merkle root that covers many records. OpenTimestamps [31] and Chainpoint [32] anchor document hashes this way. Anchoring keeps the data off chain, which matters for personal data that must not become public and permanent, and keeps the cost per record low.

G. Privacy principles in law and design

The GDPR [9] lists principles for processing personal data in Article 5. Four of them shape TrustTrail's goals: *purpose limitation* (data is collected for specified, explicit purposes), *data minimisation* (adequate, relevant and limited to what is necessary), *storage limitation* (kept no longer than necessary) and *accountability* (the controller is responsible for, and able to demonstrate, compliance). India's Digital Personal Data Protection Act, 2023 [10] applies to student data held by Indian institutes; it likewise links processing to a specified purpose and requires erasure when the purpose is no longer served. Cavoukian's *Privacy by Design* principles [33] ask that privacy be the default setting, embedded into the design, and visible and transparent to the user. The Kantara Initiative's Consent Receipt specification [34] proposes giving users a record of each consent they grant, an idea close to TrustTrail's release receipts.

Table I summarises the terms used in the rest of the paper.

III. The Base Paper and Its Gaps

TrustTrail is a follow-up to the survey by Werner, Westphall and Westphall [11], published in *Computer Networks* in 2017. This section summarises what the survey contributes, reads its tables quantitatively, and derives the gaps that TrustTrail addresses. All counts below come from the survey's own tables; we did not re-score the surveyed works.

TABLE I
TERMS USED IN THIS PAPER

Term	Meaning in TrustTrail
IdP	Keycloak, the campus OpenID Provider. Authenticates students and issues tokens.
Service	A relying party, such as the library, the LMS or the exam portal.
Profile	One of four disclosure levels: *anonymous*, *pseudonym*, *partial*, *total*. Each maps to a fixed attribute set.
Tier	A service's trust level: *fully_trusted*, *semi_trusted*, *untrusted* or *blocked*. Each tier has a profile cap.
Base tier	The tier in the service's agreement with the institute; its cap is the most the service may ever ask for.
Pairwise `sub`	The per-service pseudonym of a student.
Purpose declaration	A JWS signed by a service: attributes, purpose, deletion date (`retention_until`).
Release	One delivery of attribute values from TrustTrail to a service.
Receipt	A JWS signed by TrustTrail that describes one release.
Ledger	The append-only, hash-chained table of releases, violations and trust resets.
Violation	A refused request that breaks one of three rules; recorded in the ledger and counted against trust.
Anchor	A Merkle root of a contiguous batch of ledger entries, stored in the `TrustTrailAnchor` contract.

A. What the survey does

The survey starts from the observation that, even with IdM systems, "leakage of user data, the distribution of unnecessary attributes, and a lack of control on the dissemination of personal data are recurrent in cloud environments" [11]. It asks three research questions: which IdM model is needed to ensure privacy in cloud computing, which techniques best address the lack of privacy control in interactions, and how federation partners should agree on privacy for authentication and authorisation. To answer them, the authors ran a systematic mapping. A keyword search found 198 papers, which were filtered by title and abstract and then by content.

The central contribution is a taxonomy of privacy characteristics for IdM in the cloud, shown as a mind map in the survey's Fig. 3. It has nine characteristics, each refined into two mechanisms:

- *Transparency* (advertence, consciousness): the user is told how attributes are collected, used and disclosed.
- *Controllability* (selection, consent): the user chooses what is released and agrees to it.
- *Minimization* (control, participation): only the information needed to access a resource is released.
- *Accountability* (traceability, policies): auditing and tracking actions, "in accordance with legal requirements", so that misuse and privacy breaches can be identified and audited.
- *Data quality* (validation, confidence): attributes are correct and validated.
- *Use limitation* (obligations, policies): only authorised entities use the data, for the stated reason; a policy can state "the reason or intention for using the data" and an obligation is a directive to be carried out before or after access.
- *User-friendliness* (with or without add-ons): an interface lets users "check what data was released under what conditions, and to whom".
- *Trust* (reputation, risk): confidence in providers is assessed with metrics.
- *Obfuscation* (anonymity, pseudonym): users are not identified, or are identified only by pseudonyms.

The survey relates these characteristics to the three properties of Birrell and Schneider (confidentiality, undetectability, unlinkability) in a Venn diagram (its Fig. 4).

The survey then classifies six European research projects (PRIME, PrimeLife, ABC4Trust, GÉANT, PRISMACLOUD and CREDENTIAL; its Table 1) and 23 published proposals from 2009 to 2016 (its Table 2) by the characteristics they address. It describes five challenges for future research (its Section 4) and four works in progress by the authors' group (its Section 5 and Table 3). One of these works defines four standard *interaction profiles*, anonymous, pseudonym, partial attributes and total attributes, as OIDC scopes [35]; another adds a dissemination screen that warns the user about the provider's reputation [36].

B. Reading the tables

Table II counts how many of the surveyed proposals and projects address each characteristic. Two characteristics stand out. Accountability is marked for only 2 of the 23 proposals (Suriadi et al. [37] and Sánchez et al. [38]) and

TABLE II
COVERAGE OF THE NINE PRIVACY CHARACTERISTICS IN THE SURVEY BY WERNER ET AL. [11], COUNTED FROM ITS TABLES 1–3

Characteristic	Proposals (of 23, Table 2)	EU projects (of 6, Table 1)	Authors' works (of 4, Table 3)
Transparency	14	6	4
Controllability	17	6	3
Minimization	8	3	1
Accountability	**2**	3	**0**
Data quality	2	6	1
Use limitation	**5**	2	**0**
User-friendly	2	3	3 (all "partial")
Trust	5	2	2
Obfuscation	6	3	1

for 3 of the 6 projects. Use limitation is marked for 5 proposals and 2 projects. Data quality and user-friendliness are also rare, but they are less central to the problems described in Section I. In the authors' own works in progress, accountability and use limitation are not marked for any of the four.

The numbers suggest a pattern. Most work concentrates on the moment of disclosure: telling the user what will be released (transparency), letting the user choose (controllability) and releasing less (minimization). Far less work addresses what happens around and after the disclosure: whether the release can be traced and audited later (accountability), and whether the service's use is bound to a stated reason and time (use limitation). For students, these are exactly the missing pieces. A student who never sees a record of releases cannot exercise any control after the fact, and a promise of deletion that nobody records cannot be checked.

C. The survey's open challenges

The survey's Section 4 lists five challenges and directions for privacy-based IdM in the cloud. We summarise each and note what is still open.

Reliability assessment (Section 4.1 of the survey). Federation partners need a way to evaluate trust in each other, for example through provider reputation or risk metrics. The surveyed works that address trust mostly use reputation values or static trust levels, such as the fully trusted, semi-trusted and untrusted classes of Ferdous and Poet [2]. What is missing is a link from observed behaviour to the trust level: a provider that misbehaves keeps its level until someone changes it.

Agreement of privacy options (Section 4.2). The IdP and the service should agree in advance on the privacy features of their interaction, and the agreement should not be static. In the surveyed works, such agreements are policies or contracts outside the protocol. They are not bound to individual releases, and the service does not sign anything that could later be held against it.

Definition of interaction profiles (Section 4.3). Predefined profiles help users and services disclose the least information needed. The authors' own work defines the four profiles as OIDC scopes [35]. A profile scope says what the service *asks* for. On its own it does not stop a service from asking for a more revealing profile, and it does nothing about linkage across services through identifiers or session values.

Granular control over the dissemination of data (Section 4.4). Releasing individual attributes, not whole profiles, should be possible depending on context. This is largely solved by attribute filtering and consent screens in existing systems such as the Shibboleth extension of Orawiwattanakul et al. [39].

Ensuring the implementation of policies (Section 4.5). The survey states that "it is essential that the application of different privacy policies be guaranteed in an automated way" and calls compliance assessment "a key challenge" [11]. Sticky policies [40], [41] attach usage conditions to data, but the survey notes about one sticky-policy approach that it "does not make clear the means to ensure the effective implementation of sticky policies after receiving data".

D. Weaknesses of the surveyed approaches

Taken together with the coverage counts, the survey's own remarks point to four weaknesses in the state of the art it describes.

TABLE III
FROM THE SURVEY'S CHALLENGES TO TRUSTTRAIL'S REQUIREMENTS AND MECHANISMS

Challenge or weakness	Requirement for TrustTrail	Mechanism
Reliability assessment (4.1); static trust	Trust must follow observed behaviour and change what a service receives.	Trust score lowered by violations; tier cap enforced at Keycloak and at release (Sec. **??**).
Agreement of privacy options (4.2); purposes without enforcement	The service must commit to attributes, purpose and deletion date before any release, in a form it cannot later deny.	Signed purpose declarations; releases refused outside them; signed receipts (Sec. **??**).
Interaction profiles (4.3)	The IdP and the release service must both enforce the profile, and services must not be able to link students.	Keycloak scope assignment as tier cap; pairwise `sub`; `sid` removed by a custom mapper (Sec. **??**).
Granular control (4.4)	A service may ask for a subset of its profile.	Per-request attribute list checked against declaration and profile (Sec. **??**).
Ensuring policy implementation (4.5); no evidence trail	Every release and every rule break must be recorded so that later changes are detectable, even by the operator.	Hash-chained ledger, receipts, Merkle anchoring on Ethereum (Secs. **??**, **??**).
Proposals without implementations	The design must run on a current IdP and be measured.	Keycloak 26 implementation, 155 tests, evaluation (Secs. **??**, **??**).

1) *No evidence trail.* Accountability is the least covered characteristic. Even where it is marked, the works audit the system or trace data inside a trusted environment; none gives the user, or an outside auditor, evidence of releases that the operator cannot later alter.
2) *Purposes without enforcement.* Use limitation appears as policies and obligations, but not as a promise that is checked on every release. A service's purpose and retention period are not recorded with the data that was released under them.
3) *Static trust.* Trust levels and reputation scores are inputs set by administrators or by user feedback; a provider that breaks its promises does not automatically get less data.
4) *Proposals without implementations.* The survey itself observes of several works that they do "not provide details and [do] not validate the proposal", offer "mathematical and logical evidences, without a practical implementation", or do "not make clear how to implement" a component [11]. Several of the works are also built on SAML federations, whereas current campus deployments increasingly use OIDC.

E. From gaps to requirements

Table III maps each challenge and weakness to a requirement for TrustTrail and to the mechanism that meets it. The mechanisms are described in Sections VI and **??**, and Section **??** compares TrustTrail with the authors' works on the same nine characteristics.

One requirement from the survey we deliberately do not take on. The survey lists data quality, the validation of attribute values, as a characteristic. In a campus federation the IdP is the institute's own system of record, so we treat its attribute values as correct and do not add validation.

IV. RELATED WORK

Section III covered the works classified in the base survey. This section looks at work beyond the survey, grouped by the problem each line of work solves, and positions TrustTrail against it. Table IV summarises the comparison.

A. Privacy-preserving single sign-on

A standard OIDC provider learns every service that a user logs in to, which lets the IdP build a profile of the user's activity. Several designs remove this. SPRESSO by Fett, Küsters and Schmitz is a web SSO system in which the IdP does not learn which relying party the user logs in to [42]. Hammann, Sasse and Basin propose

a privacy-preserving extension of OIDC with the same aim [43], and Kroschewski and Lehmann address how a relying party can be authenticated in OIDC without revealing it to the IdP [44]. UPPRESSO, so far available as a preprint, aims at SSO that is both untraceable by the IdP and unlinkable across relying parties [45]. On the security side, Fett et al. gave a formal analysis of OAuth 2.0 [46], Mainka et al. systematised attacks on OIDC [47], and Wang, Chen and Wang showed through traffic analysis that deployed SSO systems had serious logic flaws [48].

These works protect the user from the IdP, or protect the protocol from attackers. TrustTrail addresses a different question: what the relying parties receive, under which promise, and whether that can be checked later. The two concerns are complementary. In a campus federation the IdP is the institute itself and already holds the student's records, so IdP tracking is less of a concern than over-release to the many services around it. We list IdP tracking as a non-goal in Section V; a deployment that needs both could combine TrustTrail with one of these protocols.

B. Anonymous credentials and self-sovereign identity

Chaum proposed in 1985 that individuals use a different pseudonym with each organisation and prove facts about themselves with credentials that cannot be linked across organisations [49]. Camenisch and Lysyanskaya gave an efficient anonymous credential system [50], which IBM implemented as Idemix [51]; Microsoft's U-Prove offers similar selective disclosure [52]. The PRIME, PrimeLife and ABC4Trust projects in the base survey built on these systems. More recently, self-sovereign identity places credentials in a wallet controlled by the user [53], [54], and the W3C Verifiable Credentials data model standardises their format [55].

Credential systems give strong unlinkability and fine-grained disclosure, often stronger than pairwise identifiers, because even the issuer cannot link presentations. They also move work to the user: the student needs a wallet, and every service needs a verifier. More importantly for our goals, a credential presentation leaves no record that the user can later show to a third party, and nothing ties the verifier's use of the data to a declared purpose. TrustTrail takes the opposite trade-off. It keeps standard OIDC on the service side and gives up some unlinkability (the institute's IdP and TrustTrail know the mapping between pseudonyms), in exchange for a central point where purpose, retention and the release record can be enforced and kept.

C. Policy-based control of disclosure

A large part of the base survey concerns policies that state how released data may be used. Sticky policies attach machine-readable usage conditions to data so that the conditions travel with it [40]. Chadwick and Fatema built a privacy-preserving authorisation system in which users' policies are stuck to their data and enforced by the cloud provider's policy decision points [41]. Orawiwattanakul et al. let users filter which attributes a Shibboleth IdP releases [39], and Suriadi, Foo and Jøsang designed a user-centric federated SSO system with privacy requirements on the attribute exchange [37]. Sánchez et al. combined privacy preferences with dynamic federation for consumer cloud services [38].

In these systems, the user or the IdP states the policy, and the service is trusted to honour it once it has the data, unless trusted hardware enforces it. TrustTrail inverts the direction of the commitment: the *service* states its purpose and retention, signs the statement with its own key, and TrustTrail checks every release against it. Neither approach controls the data once it has been released. TrustTrail adds evidence: the service's signature makes the promise attributable, and the ledger records which data was released under which promise.

D. Consent and receipts

The Kantara Initiative's Consent Receipt specification proposes that a user receive a record, in a standard format, every time they give consent, listing the controller, the purposes and the data categories [34]. TrustTrail's release receipts follow the same idea but are produced per release, signed by the release service, and linked to a ledger entry and, through anchoring, to a public chain. Research on consent interfaces explains why the record matters. Utz et al. showed in a field study that the design of GDPR consent notices strongly changes what users accept [56], and Felt et al. found that few smartphone users pay attention to, or correctly understand, permission prompts [57]. Applications also tend to request more permissions than they use [7], and OAuth is often misused in mobile applications [8]. A consent screen alone is therefore a weak control. TrustTrail's design does not rely on the student reading a prompt: the profile cap and the declaration are checked whether or not the student pays attention, and the dashboard and receipts give a record that can be read later.

E. Tamper-evident logging

Secure audit logs have a long history. Schneier and Kelsey used hash chains and evolving keys so that log entries written before a compromise cannot be changed undetectably [20]. Ma and Tsudik proposed forward-secure sequential aggregate signatures for the same purpose, with compact verification [22]. Crosby and Wallach designed history trees, in which the logger commits to the whole log after each append and auditors can check both membership of an entry and consistency between two commitments efficiently [21]. Certificate Transparency applies Merkle-tree logs at Internet scale, with monitors and auditors that check that a log only grows [24]–[26].

These systems assume an outside party that receives and keeps commitments, such as an auditor, a monitor or a gossip protocol. TrustTrail has two such parties: students, who hold receipts that contain the hash of their own entries, and a public blockchain, which holds the Merkle root of each batch. It uses the RFC 6962 leaf and node hashing and leaves consistency proofs between successive trees for future work, because the contract already refuses any batch that does not continue exactly where the previous one ended.

F. Blockchains for privacy and accountability

Zyskind, Nathan and Pentland proposed using a blockchain as an access-control manager for personal data held off chain [58]. MedRec uses Ethereum smart contracts to record permissions over medical records stored by providers [59]. Neisse, Steri and Nai-Fovino studied blockchain-based data accountability and provenance tracking with the GDPR in mind [60], and Truong et al. proposed a blockchain-based solution for GDPR-compliant personal data management [61]. Timestamping services such as OpenTimestamps and Chainpoint anchor Merkle roots of arbitrary documents in public blockchains [31], [32].

Many of the blockchain-based designs keep access rights or usage logs themselves on chain. This makes the record verifiable but raises a conflict with data protection law: information written to a public chain cannot be erased, and even pseudonymous log records can be personal data. TrustTrail writes only Merkle roots and entry ranges to the chain. A root reveals nothing about the entries it covers, and the ledger with its pseudonyms stays under the institute's control. This places TrustTrail closer to the anchoring services than to the blockchain access-control designs, with the difference that the anchored data is an accountability ledger with an enforced contiguity rule.

G. Trust and reputation

Jøsang, Ismail and Boyd survey trust and reputation systems for online services [62]. Most of them aggregate ratings from many users. In the base survey, trust appears as provider reputation shown to the user [36] and as static classes of federation membership [2]. TrustTrail's trust score is simpler and more local: it is computed only from facts that TrustTrail observes directly (registered privacy policy, registration metadata and recorded violations), and its only effect is to lower the most revealing profile the service can obtain. The weights are starting values, which Section **??** discusses.

V. Problem Statement, Threat Model and Design Goals

A. Problem statement

A university runs an OIDC identity provider for its students and a set of services of different trustworthiness. We want each service to receive only the attributes it is entitled to and has declared a purpose for, to be unable to link a student across services by identifier, and to be held to its declared purpose and retention period, as far as the institute can observe. Every release, and every observable attempt to break these rules, should leave a record that the student can inspect and that nobody, including the operator of the system, can later change without detection. A service that breaks the rules should automatically receive less in future. Services should be able to keep using standard OIDC.

B. Parties

- *Students* log in through the IdP and want to know what each service received.
- The *IdP* (Keycloak) authenticates students, holds their attribute records, and issues tokens.
- *TrustTrail* verifies declarations, releases attribute values, keeps the ledger, computes trust and anchors the ledger.

TABLE IV
TRUSTTRAIL COMPARED WITH RELATED APPROACHES (OUR READING)

Approach	Examples	What it protects	What it leaves open for our goals
Privacy profiles as OIDC scopes	[35], [36]	Less information released, chosen per interaction	Profile requested, not capped; no linkage protection; no record of releases
Privacy-preserving SSO	[42], [43], [45]	User's activity hidden from the IdP	What relying parties receive and do; no purpose binding or release record
Anonymous credentials, SSI	[51], [52], [55]	Strong unlinkability and selective disclosure	Needs wallets and verifiers; no attributable record of use or purpose
Sticky and user policies	[39]–[41]	Usage conditions attached to data	Enforcement after release; no evidence the service accepted the policy
Consent receipts	[34]	User gets a record of consent	Not linked to each release; not tamper-evident
Tamper-evident logs	[20], [21], [24]	Integrity of a log against later edits	Needs external auditors; does not decide what is released
Blockchain access control	[58], [59], [61]	Verifiable permissions and logs	Records or rights on chain (erasure conflict); new client software
TrustTrail	this paper	Profile cap, pairwise IDs, signed purpose, tamper-evident release record, trust feedback	IdP sees all logins; use after release; operator controls the pseudonym map

- *Services* are relying parties of differing trust: the library, the LMS and the examination portal in our deployment, and possibly third-party services.
- The *operator* administers TrustTrail and its database, and can reset a service's trust.
- A *verifier* is anyone who checks a receipt or the ledger against the chain: a student, an auditor or the institute's grievance officer.
- The *chain* is an Ethereum network on which the anchor contract is deployed.

C. Adversaries

We consider the following adversaries. Each is a role that a party may take, not necessarily a separate attacker.

A1 *Over-asking service.* A service requests attributes that its declaration does not cover, or a profile above the one its agreement allows.

A2 *Retaining service.* A service keeps using a declaration after its promised deletion date, for example by requesting the data again.

A3 *Colluding services.* Two or more services share what they received in order to link a student's activity across them, using identifiers, session values or authentication timestamps in the tokens.

A4 *Repudiating service.* A service later denies having promised a purpose or a deletion date, or denies having received data.

A5 *Insider with database access.* Someone with write access to TrustTrail's database file, possibly the operator, edits, deletes or reorders ledger entries, rewrites the whole chain from some entry onward, or cuts off the newest entries. We assume this adversary can remove the database's append-only triggers.

A6 *Forger.* An attacker presents a forged access token to obtain attributes, or forges a receipt to claim a release that did not happen.

D. Assumptions

- The IdP is honest and correctly configured: it authenticates students correctly, signs tokens with keys that only it holds, and grants a client only the scopes assigned to it.
- TrustTrail runs its code faithfully *at the time of a release*. Adversary A5 may tamper with its stored data afterwards, but we do not consider a TrustTrail that lies while writing an entry; Section **??** discusses this case.

TABLE V
DESIGN GOALS

Goal	Statement	Adversaries	Survey characteristic
G1	A service receives at most the attributes of the profile allowed by its current tier, and only those its declaration covers.	A1	Minimization, controllability
G2	Services cannot link a student across services through identifiers or session values in tokens.	A3	Obfuscation (pseudonym), unlinkability
G3	Every release is covered by a service-signed declaration of purpose and deletion date; requests outside the declaration or after the date are refused.	A1, A2	Use limitation
G4	The service cannot deny its declaration, and TrustTrail cannot deny a release it receipted.	A4	Accountability (policies)
G5	Any change, deletion or truncation of recorded entries is detectable, also when the operator does it.	A5	Accountability (traceability)
G6	Observed violations lower a service's trust and, through its tier, what it can obtain.	A1, A2	Trust
G7	A student can see every release and refused rule break that concerns them, and verify each one independently.	A4, A5	Transparency, user-friendliness
G8	Services use standard OIDC plus signed JSON; the added latency per login stays in the order of milliseconds.	–	(deployability)

- Each service keeps its declaration-signing key and its client secret private. A stolen key lets the thief sign in the service's name.
- The anchor contract runs on a chain that the operator cannot rewrite. A local development chain does not meet this assumption; a public network does.
- Communication uses TLS in a real deployment. Our local deployment uses plain HTTP between containers on one machine.
- Hash functions (SHA-256) and signature schemes (ECDSA P-256, RSA) are secure.

E. Design goals

Table V lists the design goals, the adversaries each goal addresses, and the survey characteristics [11] each goal corresponds to.

F. Non-goals

Some threats are outside TrustTrail's scope. We list them explicitly so the guarantees are not over-read.

- *Use after release.* Once a service holds data, TrustTrail cannot see whether it is used for the declared purpose or deleted on time. Rule 2 catches only a service that asks again.
- *IdP tracking.* Keycloak sees every login. The protocols in Section IV address this.
- *Traffic analysis.* Services that compare login times or network addresses may still correlate pseudonyms. TrustTrail removes identifiers, not timing.
- *A dishonest release service.* If TrustTrail itself omits a release from the ledger, anchoring cannot reveal an entry that was never written; only the receipt held by the student is counter-evidence.
- *Availability.* Denial-of-service against Keycloak, TrustTrail or the chain is not addressed.

VI. SYSTEM ARCHITECTURE

This section describes TrustTrail's components, the data each holds, and how a release flows through them. Section **??** then describes each mechanism in detail.

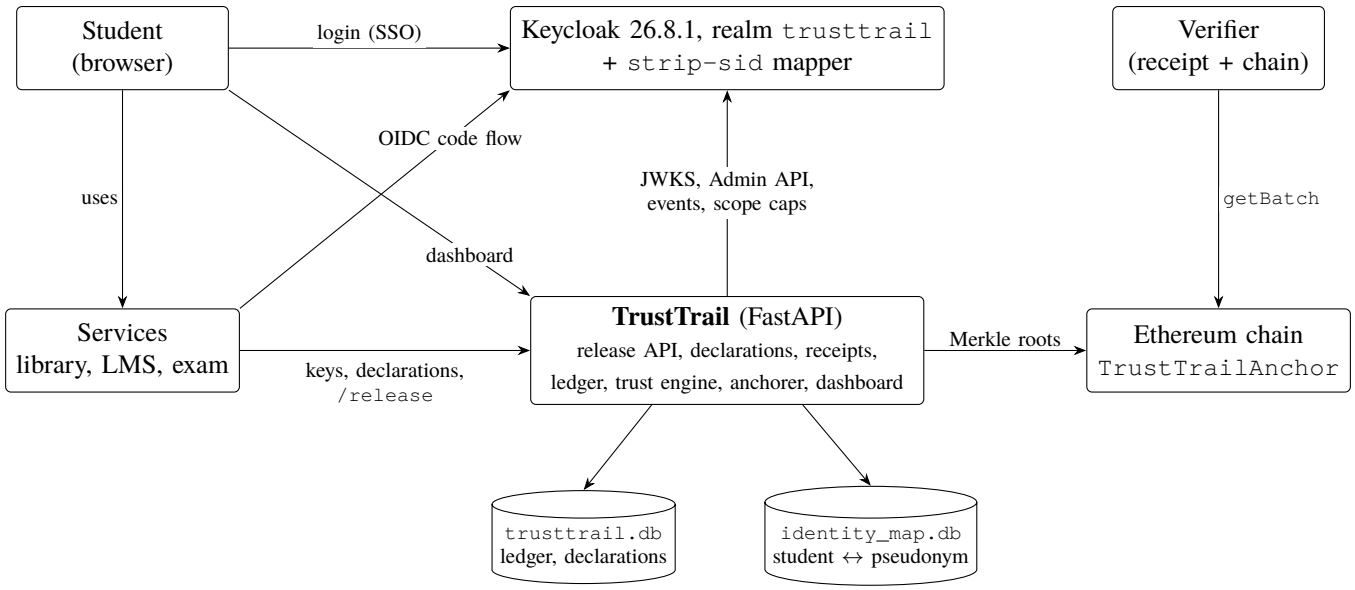


Fig. 2. TrustTrail components. Services speak standard OIDC to Keycloak and obtain attribute values only from TrustTrail. TrustTrail keeps the student-to-pseudonym map in a separate database from the ledger, and publishes only Merkle roots to the chain.

A. Design principle: the IdP decides which, TrustTrail decides what

TrustTrail splits the disclosure decision between two components. Keycloak decides *which* profile a service may obtain: a service's Keycloak client is assigned only the profile scopes up to its tier cap, so a login that asks for more fails at the IdP. The tokens that Keycloak issues to services carry a pairwise `sub`, the granted scope and no personal claims. TrustTrail decides *what* that profile contains, and it is the only path by which a service obtains attribute values: the service presents the student's access token to TrustTrail's `/release` endpoint and receives the values that its profile and its declaration allow.

We chose this split for three reasons. First, it keeps the IdP standard. Services run an ordinary OIDC authorisation code flow, and the only change to Keycloak is one small protocol mapper and configuration through its Admin API. Second, it puts every release through a single point where the declaration can be checked and the ledger written. If attributes travelled in the ID token, the IdP would release them before any purpose check, and nobody would record the release. Third, it separates the two kinds of state. Keycloak keeps the student records it already had; TrustTrail keeps declarations, the ledger and trust state, and never stores attribute values.

B. Components

Figure 2 shows the components and their connections. All run as Docker containers defined in one Compose file.

1) Keycloak: Keycloak 26.8.1 is the IdP. A seed script creates the realm `trusttrail` through the Admin REST API. The realm defines four client scopes, `profile-anonymous`, `profile-pseudonym`, `profile-partial` and `profile-total`, which carry no claim mappers: their only effect is to appear in the token's `scope` claim. Each service client has every default scope removed except `basic`, and has as optional scopes only the profiles up to its tier cap. Each service client also has two protocol mappers: Keycloak's SHA-256 pairwise subject mapper and TrustTrail's custom `trusttrail-strip-sid-mapper`, a 72-line Java class that removes the session identifier from ID and access tokens. Keycloak records only `LOGIN_ERROR` events, which TrustTrail reads to detect over-cap login attempts.

2) TrustTrail: TrustTrail is a Python 3.12 application built with FastAPI. Its 20 modules (about 2,200 lines) implement:

- the *release API*, which verifies the student's access token, the declaration and the tier cap, and returns attribute values;
- *declarations*, which verifies and stores services' signed purpose declarations and their public keys;
- *receipts*, which signs a receipt for each release with TrustTrail's ES256 key;

- the *ledger*, an append-only, hash-chained SQLite table;
- the *trust engine*, which computes each service's score and tier and applies the tier's cap to the service's Keycloak client;
- the *anchorer*, which periodically builds a Merkle tree over new ledger entries and sends its root to the contract;
- the *dashboard*, where a student logs in and sees, verifies and downloads the record of every release concerning them.

Two background threads run inside TrustTrail: the trust thread synchronises with Keycloak every 3 s, and the anchoring thread runs every 300 s by default.

3) Services: Three invented campus services, a Digital Library, a Learning System and an Examination Portal, demonstrate honest behaviour. They share one small FastAPI application (136 lines) configured by a service identifier. Three further scripted services, which we call Campus Deals, Alumni Tracker and Career Portal, each break one rule on purpose and are used in the evaluation.

4) Chain: A local Hardhat 3 node runs an in-memory Ethereum chain, and a one-shot container deploys the `TrustTrailAnchor` contract (Solidity 0.8.28) and writes its address and ABI to a deployment file that TrustTrail reads. The same contract can be deployed to the Sepolia test network by changing configuration.

C. Data stores and trust boundaries

Each party holds a different part of the state, which limits what any single compromised store reveals.

- *Keycloak* holds the student records (name, e-mail, roll number, department, programme, year, affiliation), the client definitions and each client's random pairwise salt.
- `trusttrail.db` holds service public keys, declarations, the ledger, stored receipts, trust state, cached anchor metadata and processed event identifiers. The ledger contains attribute *names*, pseudonyms and hashes, never attribute values and never a student's real user ID.
- `identity_map.db` is a separate file that maps each (service, pseudonym) pair to the Keycloak user ID. It is needed to fetch attribute values and to show a student their own entries. Keeping it apart from the ledger means the ledger can be inspected, exported or audited without revealing who is who.
- TrustTrail's receipt-signing key, an EC P-256 private key, is generated on first use into TrustTrail's data volume with file mode 0600. Each service's declaration-signing key is generated into a volume that only the services mount. No private key is stored in the source repository.
- The *chain* holds, per batch, a 32-byte root, the first and last entry numbers and a timestamp.

D. End-to-end flow

Figure 3 shows the life of one release, from a service's setup to a student's verification. The steps are:

1) *Key registration.* The service obtains a client-credentials token from Keycloak and registers its public JWK at TrustTrail.
2) *Declaration.* The service signs a purpose declaration and submits it; TrustTrail verifies and stores it and returns its identifier.
3) *Login.* The student opens the service, which redirects to Keycloak asking for `openid profile-<x>`. Keycloak authenticates the student, or reuses the SSO session, and grants the profile only if it is assigned to the client. The service exchanges the code for tokens that carry the pairwise `sub`, no `sid` and no personal claims.
4) *Release.* The service calls `POST /release` with the student's access token and the declaration identifier. TrustTrail checks the token, the declaration, the deletion date and the tier cap, maps the pseudonym to the student, fetches the attribute values from Keycloak, and in one database transaction appends a ledger entry and signs a receipt. It returns the values and the receipt.
5) *Anchoring.* At the next anchoring interval, TrustTrail computes the Merkle root of all entries since the last batch and calls `anchor(root, first, last)` on the contract.
6) *Inspection.* The student logs in to the dashboard, sees the release, runs the verification checks and downloads a receipt bundle that can be verified later against the chain alone.

If any check in step 4 fails, TrustTrail refuses the release. If the refusal breaks one of the three rules, TrustTrail writes a violation entry and the trust engine may lower the service's tier, which removes profile scopes from its Keycloak client.

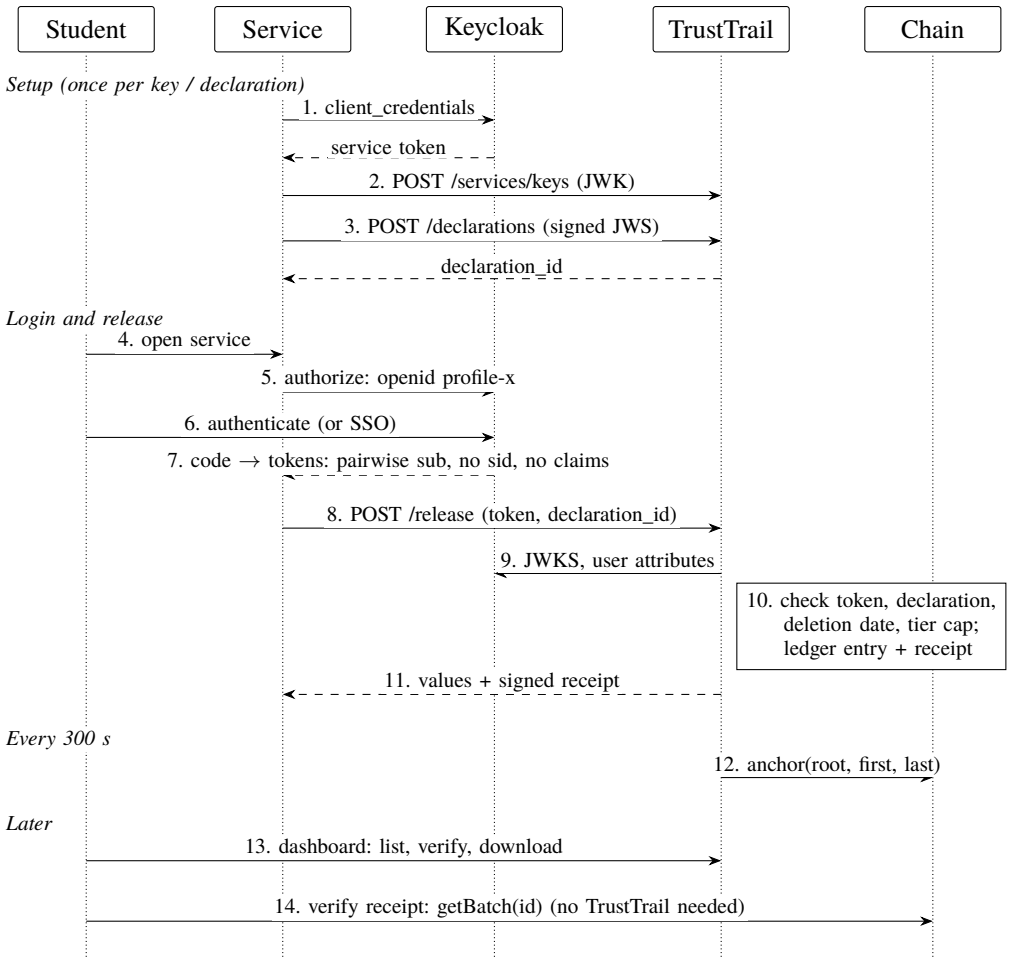


Fig. 3. Sequence of one release, from service setup to independent verification by the student.

TABLE VI
CONTAINERS IN THE DOCKER COMPOSE DEPLOYMENT

Container	Image / build	Role	Port
`keycloak`	Keycloak 26.8.1 + mapper JAR	IdP, realm `trusttrail`	8080
`chain`	Node 22, Hardhat 3	In-memory Ethereum chain	8545
`chain-deploy`	same as `chain`	Deploys `TrustTrailAnchor`, writes deployment file, exits	–
`trusttrail`	Python 3.12, FastAPI	Release API, ledger, trust, anchoring, dashboard	8000
`library`, `lms`, `exam`	Python 3.12, FastAPI	Honest demo services	8001–8003

E. Deployment

Table VI lists the containers. Keycloak's health check gates the start of TrustTrail and the services, and TrustTrail starts only after the contract has been deployed. TrustTrail's and the services' source code is mounted into their containers, so the tests and scripts run against the code in the repository.

REFERENCES

[1] E. Birrell and F. B. Schneider, "Federated identity management systems: A privacy-based characterization," *IEEE Security & Privacy*, vol. 11, no. 5, pp. 36–48, Sep. 2013.

[2] M. S. Ferdous and R. Poet, "Managing dynamic identity federations using security assertion markup language," *J. Theor. Appl. Electron. Commer. Res.*, vol. 10, no. 2, pp. 53–76, May 2015.

[3] S. Cantor, J. Kemp, R. Philpott, and E. Maler, "Assertions and protocols for the OASIS security assertion markup language (SAML) V2.0," OASIS Standard. http://docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf, Mar. 2005, accessed: Nov. 10, 2026.

[4] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect core 1.0 incorporating errata set 2," OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html, 2023, accessed: Nov. 10, 2026.

[5] D. Hardt, "The OAuth 2.0 authorization framework," IETF, RFC 6749, Oct. 2012.

[6] *Keycloak Documentation*, https://www.keycloak.org/documentation, Keycloak / Cloud Native Computing Foundation, accessed: Nov. 10, 2026.

[7] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner, "Android permissions demystified," in *Proc. 18th ACM Conf. Comput. Commun. Secur. (CCS)*, Oct. 2011, pp. 627–638.

[8] E. Y. Chen, Y. Pei, S. Chen, Y. Tian, R. Kotcher, and P. Tague, "OAuth demystified for mobile application developers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2014, pp. 892–903.

[9] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)," Official Journal of the European Union, L 119, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj, Apr. 2016, accessed: Nov. 10, 2026.

[10] Government of India, "The digital personal data protection act, 2023 (no. 22 of 2023)," The Gazette of India, Extraordinary, Part II, Sec. 1, Aug. 11, 2023. https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf, 2023, accessed: Nov. 10, 2026.

[11] J. Werner, C. M. Westphall, and C. B. Westphall, "Cloud identity management: A survey on privacy strategies," *Computer Networks*, vol. 122, pp. 29–42, Jul. 2017.

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON web signature (JWS)," IETF, RFC 7515, May 2015.

[13] A. Pfitzmann and M. Hansen, "A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management," Version v0.34, TU Dresden and ULD Kiel. https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf, Aug. 2010, accessed: Nov. 10, 2026.

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," IETF, RFC 7519, May 2015.

[15] D. Temoshok, J. P. Richer, Y.-Y. Choong, J. Fenton, N. Lefkovitz, A. Regenscheid, and R. Galluzzo, "Digital identity guidelines: Federation and assertions," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-63C-4, Jul. 2025.

[16] M. Jones, "JSON web algorithms (JWA)," IETF, RFC 7518, May 2015.

[17] ——, "JSON web key (JWK)," IETF, RFC 7517, May 2015.

[18] M. Jones and N. Sakimura, "JSON web key (JWK) thumbprint," IETF, RFC 7638, Sep. 2015.

[19] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *J. Cryptol.*, vol. 3, no. 2, pp. 99–111, 1991.

[20] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.

[21] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Security Symp.*, 2009, accessed: Nov. 10, 2026.

[22] D. Ma and G. Tsudik, "Extended abstract: Forward-secure sequential aggregate authentication," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2007, pp. 86–91.

[23] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology — CRYPTO '87*, ser. Lecture Notes in Computer Science, vol. 293. Springer, 1988, pp. 369–378.

[24] B. Laurie, "Certificate transparency," *Commun. ACM*, vol. 57, no. 10, pp. 40–46, Oct. 2014.

[25] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," IETF, RFC 6962, Jun. 2013.

[26] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," IETF, RFC 9162, Dec. 2021.

[27] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008, accessed: Nov. 10, 2026.

[28] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger (shanghai version)," Ethereum Yellow Paper. https://ethereum.github.io/yellowpaper/paper.pdf, 2025, accessed: Nov. 10, 2026.

[29] *Solidity Documentation*, https://docs.soliditylang.org/, Solidity Team, Ethereum Foundation, accessed: Nov. 10, 2026.

[30] *Hardhat Documentation*, https://hardhat.org/docs, Nomic Foundation, accessed: Nov. 10, 2026.

[31] P. Todd, "OpenTimestamps: Scalable, trust-minimized, distributed timestamping with Bitcoin," https://petertodd.org/2016/opentimestamps-announcement, Sep. 2016, accessed: Nov. 10, 2026.

[32] Tierion, "Chainpoint: A blockchain proof and anchoring standard," https://chainpoint.org/, accessed: Nov. 10, 2026.

[33] A. Cavoukian, "Privacy by design: The 7 foundational principles — implementation and mapping of fair information practices," Information and Privacy Commissioner of Ontario (mirror). https://student.cs.uwaterloo.ca/~cs492/papers/7foundationalprinciples_longer.pdf, 2011, accessed: Nov. 10, 2026.

[34] M. Lizar and D. Turner, "Consent receipt specification, version 1.1.0," Kantara Initiative. https://kantara.atlassian.net/wiki/x/Noo1, Feb. 2018, accessed: Nov. 10, 2026.

[35] J. Werner, C. M. Westphall, G. Weingartner, G. A. Geronimo, and C. B. Westphall, "An approach to IdM with privacy in the cloud," in *Proc. IEEE Int. Conf. Comput. Inf. Technol.; Ubiquitous Comput. Commun.; Dependable, Autonomic Secure Comput.; Pervasive Intell. Comput. (CIT/IUCC/DASC/PICOM)*, Oct. 2015, pp. 168–175.

[36] J. Werner and C. M. Westphall, "A model for identity management with privacy in the cloud," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2016, pp. 463–468.

[37] S. Suriadi, E. Foo, and A. Jøsang, "A user-centric federated single sign-on system," *J. Netw. Comput. Appl.*, vol. 32, no. 2, pp. 388–401, Mar. 2009.

[38] R. Sanchez, F. Almenares, P. Arias, D. Diaz-Sanchez, and A. Marin, "Enhancing privacy and dynamic federation in IdM for consumer cloud computing," *IEEE Trans. Consum. Electron.*, vol. 58, no. 1, pp. 95–103, Feb. 2012.

[39] T. Orawiwattanakul, K. Yamaji, M. Nakamura, T. Kataoka, and N. Sonehara, "User-controlled privacy protection with attribute-filter mechanism for a federated SSO environment using Shibboleth," in *Proc. Int. Conf. P2P, Parallel, Grid, Cloud Internet Comput. (3PGCIC)*, Nov. 2010, pp. 243–249.

[40] S. Pearson and M. Casassa-Mont, "Sticky policies: An approach for managing privacy across multiple parties," *Computer*, vol. 44, no. 9, pp. 60–68, Sep. 2011.

[41] D. W. Chadwick and K. Fatema, "A privacy preserving authorisation system for the cloud," *J. Comput. Syst. Sci.*, vol. 78, no. 5, pp. 1359–1373, Sep. 2012.

[42] D. Fett, R. Küsters, and G. Schmitz, "SPRESSO: A secure, privacy-respecting single sign-on system for the web," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2015, pp. 1358–1369.

[43] S. Hammann, R. Sasse, and D. Basin, "Privacy-preserving OpenID Connect," in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur. (ASIA CCS)*, Oct. 2020, pp. 277–289.

[44] M. Kroschewski and A. Lehmann, "Save the implicit flow? Enabling privacy-preserving RP authentication in OpenID Connect," in *Proc. Privacy Enhancing Technol. (PoPETs)*, vol. 2023, no. 4, Oct. 2023, pp. 96–116.

[45] C. Guo, J. Lin, Q. Cai, W. Wang, W. Zhu, J. Jing, Q. Wang, B. Zhao, and F. Li, "UPPRESSO: Untraceable and unlinkable privacy-preserving single sign-on services," arXiv:2110.10396 [cs.CR]. https://arxiv.org/abs/2110.10396, 2021, accessed: Nov. 10, 2026.

[46] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2016, pp. 1204–1215.

[47] C. Mainka, V. Mladenov, J. Schwenk, and T. Wich, "SoK: Single sign-on security — an evaluation of OpenID Connect," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Apr. 2017, pp. 251–266.

[48] R. Wang, S. Chen, and X. Wang, "Signing me onto your accounts through Facebook and Google: A traffic-guided security study of commercially deployed single-sign-on web services," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2012, pp. 365–379.

[49] D. Chaum, "Security without identification: Transaction systems to make Big Brother obsolete," *Commun. ACM*, vol. 28, no. 10, pp. 1030–1044, Oct. 1985.

[50] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology — EUROCRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2045. Springer, 2001, pp. 93–118.

[51] J. Camenisch and E. V. Herreweghen, "Design and implementation of the idemix anonymous credential system," in *Proc. 9th ACM Conf. Comput. Commun. Secur. (CCS)*, Nov. 2002, pp. 21–30.

[52] C. Paquin and G. Zaverucha, "U-Prove cryptographic specification V1.1 (revision 3)," Microsoft Corporation, Tech. Rep., Dec. 2013, https://www.microsoft.com/en-us/research/project/u-prove/. Accessed: Nov. 10, 2026.

[53] C. Allen, "The path to self-sovereign identity," Life With Alacrity (blog). https://www.lifewithalacrity.com/2016/04/the-path-to-self-soverereign-identity.html, Apr. 2016, accessed: Nov. 10, 2026.

[54] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," *Comput. Sci. Rev.*, vol. 30, pp. 80–86, Nov. 2018.

[55] W3C, "Verifiable credentials data model v2.0," W3C Recommendation. https://www.w3.org/TR/vc-data-model-2.0/, May 2025, accessed: Nov. 10, 2026.

[56] C. Utz, M. Degeling, S. Fahl, F. Schaub, and T. Holz, "(Un)informed consent: Studying GDPR consent notices in the field," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2019, pp. 973–990.

[57] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner, "Android permissions: User attention, comprehension, and behavior," in *Proc. 8th Symp. Usable Privacy Secur. (SOUPS)*, Jul. 2012, pp. 1–14.

[58] G. Zyskind, O. Nathan, and A. Pentland, "Decentralizing privacy: Using blockchain to protect personal data," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2015, pp. 180–184.

[59] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using blockchain for medical data access and permission management," in *Proc. 2nd Int. Conf. Open Big Data (OBD)*, Aug. 2016, pp. 25–30.

[60] R. Neisse, G. Steri, and I. Nai-Fovino, "A blockchain-based approach for data accountability and provenance tracking," in *Proc. 12th Int. Conf. Availability, Reliability Secur. (ARES)*, Aug. 2017, pp. 1–10.

[61] N. B. Truong, K. Sun, G. M. Lee, and Y. Guo, "GDPR-compliant personal data management: A blockchain-based solution," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1746–1761, 2020.

[62] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decis. Support Syst.*, vol. 43, no. 2, pp. 618–644, Mar. 2007.